

ENGLISH ERA

Autora: Dubauskaitė Miglė

Tutor: Javier Luque

Centro de formación: Core Networks Sevilla

Fecha de presentación: 15/06/2026



ÍNDICE

0.	Resumen	4
0.	Abstract.....	5
1.	Introducción	6
1.1	Contexto y motivación	6
1.2	Objetivos generales y específicos.....	7
1.3	Estructura del documento.....	7
2.	Descripción del Proyecto.....	8
2.1	Descripción detallada del proyecto	8
2.2	Objetivos específicos del proyecto	9
3.	Marco Teórico y Referencial.....	10
3.1	Conceptos teóricos y técnicos necesarios	10
3.2	Normativa y estándares aplicables	12
4.	Metodología	13
4.1	Métodos y técnicas utilizadas.....	13
4.2	Herramientas y recursos empleados	13
5.	Desarrollo del Proyecto	14
5.1	Planificación.....	14
5.2	Diseño.....	15
5.2.1	Diseño de la arquitectura	15
5.2.2	Diseño del modelo de datos	16
5.3	Implementación	18
5.3.1	Backend: API REST con Spring Boot	18
5.3.2	Frontend: React con TypeScript	19
5.3.3	Aplicación móvil: React Native con Expo.....	20
5.4	Pruebas.....	21
6.	Resultados y Producto Final	22
6.1	Resultados obtenidos	22
6.2	Descripción del producto final	23
7.	Evaluación y Discusión.....	24

7.1	Evaluación según objetivos	24
7.2	Discusión de aciertos y áreas de mejora	24
7.3	Comparación con expectativas iniciales y estudios previos.....	25
8.	Conclusiones.....	26
8.1	Principales hallazgos y logros	26
8.2	Limitaciones del proyecto	27
8.3	Recomendaciones.....	27
9.	Bibliografía.....	28
10.	Anexos	30

0. Resumen

Este trabajo de fin de grado recoge todo el proceso de diseño, desarrollo e implementación de English Era, una plataforma digital diseñada específicamente para que la gente aprenda inglés de forma efectiva. Los idiomas siempre han sido un puente entre distintas poblaciones, permitiendo la comunicación sin importar de dónde se encuentre uno. Según la base de datos especializada en lenguas del mundo, Ethnologue, el idioma más hablado en el planeta es el inglés, lo que hace fundamental su aprendizaje a escala mundial para los no nativos. Sin embargo, la mayoría de las plataformas existentes se basan en contenido estático y prefabricado, disperso y desorganizado, sin integrar práctica estructurada, recursos auténticos y comunidad en un único entorno.

English Era responde a esta necesidad. Se propuso crear una experiencia de aprendizaje que fuera realmente completa y motivadora, combinando práctica interactiva de gramática y vocabulario a través de cuestionarios, con acceso a recursos educativos diferentes noticias en inglés, letras de canciones, documentos en PDF y vídeos, una comunidad activa donde los estudiantes pueden compartir publicaciones y descubrir eventos, y un asistente conversacional basado en el modelo Llama 3.3 de Meta, integrado mediante la API de Groq, para resolver dudas al instante.

Desde el punto de vista técnico, se ha estructurado la plataforma en tres componentes bien definidos: un backend construido con Spring Boot que expone una API REST, un frontend web hecho con React y TypeScript, y una aplicación móvil desarrollada con React Native y Expo. Ambas interfaces - la web y la móvil - consumen exactamente los mismos servicios del servidor, garantizando que los datos y la lógica de negocio siempre estén alineados. La autenticación funciona con tokens JWT y el sistema de progresión sigue fielmente los niveles del Marco Común Europeo de Referencia para las Lenguas, desde A1 hasta C2.

0. Abstract

This TFG's project documents the full design, development and deployment process of English Era, a digital platform built to support effective English language learning. The platform combines four core elements within a single environment: interactive grammar, vocabulary, idioms and text comprehension quizzes, a curated library of authentic educational resources including current news articles, song lyrics, PDF documents and videos, a community space where learners can share posts and discover events, and an AI-powered conversational assistant based on Meta's Llama 3.3 model, integrated via the Groq API, for instant grammar and vocabulary support.

English is the most widely spoken language in the world according to Ethnologue, making its acquisition a globally relevant skill for non-native speakers. Existing platforms, however, tend to rely on static or prefabricated content and rarely integrate real-world resources, community features and intelligent assistance in a unified experience. English Era addresses this gap.

Technically, the platform is structured around three components: a Spring Boot backend exposing a REST API, a React and TypeScript web frontend, and a React Native mobile application built with Expo. Both client interfaces consume the same server endpoints, ensuring consistent data and business logic across devices. Session management is handled through JWT tokens, and the progression system follows the six proficiency levels defined by the Common European Framework of Reference for Languages, from A1 to C2.

English Era demonstrates that a fully functional, multi-platform educational application can be built and deployed using open-source technologies and low-cost cloud services, lowering the barrier to entry for modern educational software development.

1. Introducción

1.1 Contexto y motivación

En la actualidad, el idioma extranjero, sobre todo el inglés, se ha convertido en imprescindible en casi cualquier contexto académico, profesional y privado. De este modo, a lo largo de los años, las herramientas digitales diseñadas para la enseñanza no han dejado de crecer. Sin embargo, la mayoría de estas plataformas se basan en contenido estático y prefabricado, que rápidamente hace perder la motivación a las personas con necesidad o ganas de aprender un idioma nuevo. Por otro lado, en general, en internet hay una enorme cantidad de contenido real en inglés: noticias actuales, diversos videos en YouTube (de aprendizaje, letras de canciones, recortes de películas o series, videos interesantes) y documentos variados que sirven como valiosos recursos educativos. Lo único es que, como se puede apreciar, el contenido está completamente disperso y desorganizado, lo que dificulta encontrar algo útil según el nivel del estudiante. Todavía no existe ninguna aplicación que estructure y organice todo el contenido disponible online según los niveles del Marco Común Europeo de Referencia, para que cualquier persona pueda acceder de forma sencilla al material didáctico más apropiado para su nivel de competencia, sin requerir mucho tiempo de búsqueda en internet.

A esto se suma otro problema que la mayoría de las aplicaciones no abordan: la posibilidad de practicar inglés en un contexto social real. Responder preguntas en la web de las pruebas no es suficiente para comunicarse en la vida real. Los usuarios necesitan un espacio donde puedan expresarse con confianza e interactuar con otras personas sin la angustia de ser evaluados o juzgados. Dicha dimensión comunitaria, con eventos en vivo como charlas, clubes de lectura o sesiones de conversación, es bastante difícil de encontrar en otras plataformas de aprendizaje de idiomas.

Estas tres ideas fueron exactamente lo que motivó la creación de la aplicación de English Era. El objetivo era crear algo que fuera robusto y que tuviera una base pedagógica sólida, aprovechando las herramientas tecnológicas disponibles.

La decisión sobre qué tecnologías usar tuvo, además, un propósito de aprendizaje. Trabajar en este proyecto permitió profundizar y reforzar mis conocimientos en

arquitecturas cliente-servidor, bases de datos, uso de Android Studio, apk, ngrok, sistemas de autenticación seguros, integración de servicios externos, etc. - todas competencias que son directamente valiosas en el mercado laboral actual.

1.2 Objetivos generales y específicos

El propósito principal es diseñar, construir y lanzar una plataforma educativa en línea que funcione de verdad, facilitando que la gente aprenda inglés a través de diferentes formas de práctica y recursos variados, accesible desde cualquier navegador web o dispositivo móvil.

Se han marcado varios objetivos que se quieren cumplir. Se ha establecido:

- Crear un sistema de autenticación seguro usando JWT tokens que maneje tanto sesiones cortas como de larga duración de forma fluida.
- Desarrollar una sección de cuestionarios interactivos bien organizados por nivel MCER y divididos en categorías prácticas gramática, vocabulario, comprensión de texto, y modismos (*idioms*), con un sistema que acumula puntos de experiencia y refleja el progreso del usuario hacia el siguiente nivel MCER.
- Conectar la API de NewsAPI.org para proporcionar artículos de noticias reales en inglés como material de lectura auténtico que los usuarios encuentren relevante.
- Integrar el modelo Llama 3.3 mediante la API de Groq para ofrecerle a los usuarios un asistente conversacional al instante que entienda de aprendizaje de inglés.
- Construir un área comunitaria donde los usuarios puedan compartir contenido, comentar y participar en eventos.
- Asegurar que todo funcione de manera consistente entre la versión web y la aplicación móvil, usando la misma API REST como base.

1.3 Estructura del documento

El presente documento se organiza en diez secciones. La sección 2 describe el proyecto en detalle y sus objetivos específicos. La sección 3 recoge el marco teórico y referencial, incluyendo los conceptos técnicos empleados y la normativa aplicable. La sección 4 explica la metodología seguida durante el desarrollo, tanto los métodos utilizados como las herramientas empleadas. La sección 5 constituye el núcleo técnico del documento,

detallando la planificación, el diseño de la arquitectura y el modelo de datos, y la implementación de los tres componentes de la plataforma: el backend, el frontend web y la aplicación móvil, junto con las pruebas realizadas. La sección 6 presenta los resultados obtenidos y describe el producto final. La sección 7 evalúa el proyecto según los objetivos iniciales y discute los aciertos y áreas de mejora. Finalmente, la sección 8 recoge las conclusiones, limitaciones y recomendaciones para el trabajo futuro.

2. Descripción del Proyecto

2.1 Descripción detallada del proyecto

English Era es una plataforma de aprendizaje del inglés que funciona a través de tres capas: un servidor backend, una aplicación web y una aplicación móvil, todas funcionando en conjunto para ofrecer una experiencia consistente sin importar desde qué dispositivo se acceda.

La parte más importante está desarrollada en el backend. Se ha creado con Spring Boot, y allí se encuentra toda la lógica del negocio con los datos expuestos mediante una API REST. El servidor se encarga de gestionar la autenticación, los cuestionarios, los recursos didácticos, las publicaciones, los eventos y las noticias en tiempo real, además de coordinar la comunicación con el asistente de IA (con Groq).

La interfaz web, desarrollada con React y TypeScript, permite registrarse, iniciar sesión y practicar con cuestionarios que se adaptan a nivel de cada usuario. También permite consultar recursos organizados por dificultad, contribuir a la comunidad, explorar eventos próximos y mantener conversaciones con el asistente. Para mantener sesión segura, el token de autenticación se guarda en *local storage*.

La aplicación móvil, desarrollada con React Native y Expo, trae la misma funcionalidad de la versión web, pero pensada específicamente para dispositivos Android y iOS, aunque el desarrollo y las pruebas se realizaron principalmente sobre Android mediante emulador y dispositivo físico. La navegación funciona con pestañas en la parte inferior, tal como es habitual acostumbrados a ver en las aplicaciones móviles. El token de

sesión se resguarda utilizando el módulo de almacenamiento cifrado que ofrece Expo, garantizando mayor seguridad.

La base de datos principal es MySQL, gestionada con Spring Data JPA e Hibernate como ORM. En ella se almacenan los datos estructurados de la plataforma: usuarios, cuestionarios, lecciones, publicaciones, comentarios, eventos, logros y el progreso de cada usuario. La plataforma incluye además un panel de administración accesible únicamente para usuarios con rol administrador, desde el que se puede gestionar usuarios, preguntas, publicaciones y comentarios.

Además, está integrada el sistema de progresión que sigue los niveles del MCER: A1, A2, B1, B2, C1 y C2. Cada respuesta correcta otorga 10 puntos de experiencia, y la plataforma refleja el progreso del usuario mediante una barra de XP que muestra el avance hacia el siguiente nivel MCER.

2.2 Objetivos específicos del proyecto

Los objetivos específicos del proyecto se organizan en torno a cuatro áreas clave: arquitectura y seguridad, objetivos pedagógicos, integración externa y despliegue y operabilidad.

En primer lugar, en materia de arquitectura y seguridad, se propuso diseñar una API REST sólida y bien estructurada utilizando Spring Boot, implementar un sistema robusto de autenticación y autorización basado en JWT que contemple tanto sesiones persistentes como temporales, y configurar CORS de manera apropiada.

Por otro lado, desde la perspectiva pedagógica, se estructuró el sistema de niveles de acuerdo con el Marco Común Europeo de Referencia, crear un banco de preguntas organizadas por nivel y tipo de dificultad, desarrollar la lógica acorde que gestione la puntuación y el ascenso de nivel de los usuarios, y poner a disposición un conjunto variado de recursos didácticos clasificados según el grado de complejidad.

En cuanto a la integración externa, la plataforma se conectó con NewsAPI.org para traer titulares recientes y actualizados en inglés, así como integrar la API de Groq, aprovechando el poder de Llama 3.3, para alimentar un asistente conversacional inteligente.

Respecto al despliegue, todos los servicios se contenerizaron mediante Docker y Docker Compose, lo que permite levantar el entorno completo — backend, frontend y base de datos — con un único comando en cualquier máquina con Docker instalado. La aplicación móvil se distribuyó de dos formas: mediante Expo Go con túnel ngrok para pruebas en red móvil, y mediante una APK generada con Expo e instalada directamente en dispositivo físico Android. El proyecto incluye además configuración semi preparada para despliegue en Render.com (backend) y Netlify (frontend) como línea de evolución futura.

3. Marco Teórico y Referencial

3.1 Conceptos teóricos y técnicos necesarios

En English Era se han implementado diversos conceptos teóricos y técnicas.

La arquitectura de cliente-servidor que funciona a través de una API REST permite que el frontend web y la aplicación móvil consuman exactamente los mismos servicios del backend de forma independiente, sin duplicar lógica de negocio y garantizando que los datos sean siempre consistentes en ambas plataformas.

Spring Boot actúa como la base fundamental del servidor, e integra varios módulos especializados: Spring Web para construir los endpoints REST, Spring Security para gestionar autenticación y autorización mediante JWT de forma robusta, y Spring Data JPA para simplificar el acceso a la base de datos relacional. Esta combinación permitió construir un backend robusto y bien estructurado sin necesidad de configuración manual de cada componente.

La autorización por roles se gestiona mediante Spring Security. Solo los usuarios con rol `ROLE_ADMIN` pueden acceder al panel de administración y crear, editar o eliminar

eventos. Los endpoints de consulta de publicaciones, lecciones, noticias y eventos son públicos y no requieren token.

El sistema genera tokens de duración variable: un día para sesiones estándar y un año cuando el usuario activa la opción de recordar sesión, lo que permite adaptar la experiencia según las preferencias del usuario sin comprometer la seguridad.

El frontend de English Era se organiza en nueve páginas: registro, login, inicio, cuestionarios, recursos, publicaciones, eventos, perfil y chat. Dichas páginas han sido construidas como componentes React que consumen la API del backend y reaccionan en tiempo real a las acciones del usuario.

El uso de React Native permitió reutilizar la lógica de negocio del frontend web y adaptarla a los patrones de interacción móvil, como, por ejemplo, la navegación por pestañas o almacenamiento seguro del token con expo-secure-store, sin tener que desarrollar una aplicación nativa desde cero para cada plataforma.

La plataforma utiliza MySQL, que funciona como el sistema relacional principal, organizando la información en tablas con esquemas definidos previamente y garantizando la integridad referencial mediante claves foráneas y transacciones ACID que aseguran consistencia. Aquí es donde reside la información estructurada: datos de usuarios, cuestionarios, lecciones, eventos y progreso del aprendizaje.

English Era aprovecha el modelo Llama 3.3 70B, desarrollado por Meta y disponible mediante la API de Groq. Este modelo actúa como un asistente conversacional especializado que guía a los estudiantes en el aprendizaje del inglés. Puede responder preguntas sobre gramática, vocabulario, pronunciación y modismos siempre manteniendo un tono pedagógico que se adapta al nivel de cada usuario. La integración se realiza a través del servicio GroqService en el backend, que construye cada petición con un prompt de sistema diseñado específicamente para el contexto educativo, permitiendo además que el asistente responda en español cuando el usuario lo necesita.

Por último, el archivo docker-compose.yml orquesta cinco servicios —MySQL, MongoDB, el backend Spring Boot, el frontend Nginx y Adminer— de forma que cualquier

desarrollador puede levantar el entorno completo con un único comando, eliminando los problemas de configuración entre máquinas.

3.2 Normativa y estándares aplicables

El desarrollo de la aplicación de English Era siguió varias normativas y estándares.

En primer lugar, se ha utilizado la normativa de Marco Común Europeo de Referencia. El Marco Común Europeo de Referencia para las Lenguas, creado por el Consejo de Europa, ofrece una base sólida para que toda Europa desarrolle programas de lenguas, diseñe sus currículos y establezca sistemas de evaluación coherentes. El MCER divide el aprendizaje en seis niveles agrupados en tres categorías bien definidas: los usuarios básicos (A1 y A2), que están dando sus primeros pasos en el idioma; los usuarios independientes (B1 y B2), quienes pueden desenvolverse sin dificultad en la mayoría de las situaciones del día a día; y los usuarios competentes (C1 y C2), capaces de expresarse con soltura en contextos especializados o de alta exigencia profesional. Además, ayuda a los usuarios a comprender claramente su progreso porque es el estándar en los países aceptado en certificaciones de Cambridge, IELTS y TOEFL, así como en procesos de selección y desarrollo profesional. English Era ha adoptado la división de MCER como estrategia de progresión: todos los cuestionarios, recursos didácticos y el perfil mismo del usuario giran en torno a estos seis niveles.

En segundo lugar, la seguridad es un aspecto fundamental en cualquier sistema que gestione información sensible, como datos personales y credenciales de acceso. English Era sigue las prácticas recomendadas por OWASP para el desarrollo de aplicaciones web. Las contraseñas se almacenan cifradas mediante BCrypt, estándar ampliamente adoptado por su resistencia a ataques de fuerza bruta. La gestión de sesiones se realiza mediante tokens JWT firmados con HMAC-SHA256, eliminando la necesidad de mantener estado en el servidor. El acceso a la base de datos se realiza a través de Spring Data JPA, que previene las inyecciones SQL mediante consultas parametrizadas.

En tercer lugar, se utiliza el diseño de API REST, que sigue los principios que Roy Fielding. Se integraron los métodos HTTP GET, POST, PUT, PATCH y DELETE,

comunicando el resultado de cada operación mediante los códigos de estado HTTP correspondientes.

4. Metodología

4.1 Métodos y técnicas utilizadas

English Era se desarrolló siguiendo una metodología iterativa e incremental, basada en principios ágiles que priorizan la flexibilidad y la adaptación constante. Antes de empezar a escribir código se planificó meticulosamente la estructura del sistema con diagramas de componentes, diagramas de entidad-relación para la base de datos y búsqueda de bocetos de las interfaces de usuario. Este enfoque permitió identificar los problemas de diseño más rápido, ajustar qué era prioritario conforme avanzaba el trabajo y mantener siempre una versión operativa sobre la cual seguir construyendo.

Para organizar el trabajo se utilizó Notion, donde se definían los objetivos de cada semana y se registraba el progreso de cada fase. El control de versiones del código se gestionó con Git a través de un repositorio en GitHub.

Durante todo el desarrollo, las pruebas fueron una actividad constante y fundamental. Se combinaron verificaciones manuales de las funcionalidades que iban implementándose con pruebas de la API utilizando Postman. Se enfocó principalmente en validar los flujos de autenticación, la lógica de puntuación y progresión de nivel, y la integración con los servicios externos, asegurando así que todo funcionara según se esperaba.

4.2 Herramientas y recursos empleados

Para el desarrollo del backend en Java se utilizó el Spring Tools for Eclipse. Fue elegido por su estabilidad, su facilidad de integración con servicios empresariales y la abundante documentación disponible.

En cuanto al frontend web y la aplicación móvil, se optó por Visual Studio Code porque es un editor de código ligero con terminal integrada, soporte nativo para Git y un amplio catálogo de extensiones para JavaScript, TypeScript y React Native. Fue elegido por su amplio catálogo de extensiones, su terminal integrada y su soporte nativo para Git.

Se implementaron React JS y React Native porque ofrecen gran flexibilidad, rendimiento y ecosistema robusto, con posibilidad de crear interfaces de usuario dinámicas y mantener el código organizado mediante componentes.

Para gestionar las dependencias del backend se utilizó Apache Maven, mientras que en el frontend web y móvil se empleó npm como gestor de paquetes. Como herramienta de construcción, el frontend web utilizó Vite y la aplicación móvil el CLI de Expo. Para la aplicación móvil se utilizaron además expo-secure-store para el almacenamiento cifrado del token de sesión y expo-file-system para la persistencia local del historial de cuestionarios.

Para la navegación entre páginas del frontend web se utilizó React Router, y para el diseño visual, Tailwind CSS.

5. Desarrollo del Proyecto

5.1 Planificación

La planificación se abordó en dos escalas.

A nivel macro, se definieron las grandes fases del desarrollo que se quieren conseguir. Entre ellas, es el diseño de la arquitectura y el modelo de datos, implementación del backend, desarrollo del frontend web, desarrollo de la aplicación móvil, integración de servicios externos y pruebas de integración.

A nivel micro, se definieron tareas semanales con objetivos concretos que se documentaron en Notion, lo que permitió mantener el foco en cada iteración y ajustar prioridades conforme avanzaba el desarrollo.

La fase de diseño se centró en definir el modelo de datos, especificar los endpoints de la API y elaborar referencias visuales y bocetos de las interfaces. El modelo de datos se diseñó prestando atención a la normalización de las tablas relacionales y a las relaciones entre entidades: usuarios y su progreso, publicaciones y comentarios, usuarios y logros.

5.2 Diseño

5.2.1 Diseño de la arquitectura

La arquitectura del backend sigue el patrón de capas clásico que se encuentra en las aplicaciones Java. Los controladores REST constituyen la capa de presentación, encargándose de recibir las peticiones HTTP, delegar la lógica de negocio a la capa de servicios y devolver las respuestas en formato JSON. La capa de negocio está compuesta por los servicios, que implementan las reglas específicas de la aplicación y coordinan cómo se accede a los datos. Finalmente, en la capa de datos se encuentran los repositorios creados, que abstraen las complejidades del acceso a la base de datos utilizando principalmente Spring Data JPA para MySQL, con Spring Data MongoDB presente como artefacto residual del diseño inicial, pero sin uso activo en la versión final, donde toda la persistencia recae sobre MySQL.

En cuanto al frontend web, la arquitectura se estructura en torno al modelo de componentes de React. La aplicación está organizada en páginas cada una conectada a una ruta mediante React Router. Los elementos comunes, como barra de navegación, pie de página y widget de chat, se encapsulan en componentes reutilizables compartidos entre páginas. La comunicación con el backend se centraliza en un módulo cliente único que se encarga de gestionar la inyección del token de autorización y manejar los errores de autenticación de manera consistente. La aplicación incluye además una página de administración accesible únicamente para usuarios con rol administrador, desde la que se gestionan usuarios, preguntas, publicaciones y comentarios mediante un panel con pestañas.

La aplicación móvil organiza la navegación en dos niveles diferenciados. Un navegador de pila gestiona el flujo de autenticación, mostrando las pantallas de

registro e inicio de sesión a los usuarios no autenticados. Una vez autenticado, un navegador de pestañas inferiores proporciona acceso directo a las siete pestañas principales: Home, Quiz, Events, Blog, Resources, Profile y Admin, esta última visible únicamente para usuarios con rol administrador. El estado de la sesión se gestiona de forma global mediante React Context.

5.2.2 Diseño del modelo de datos

La base de datos relacional MySQL está compuesta por once tablas principales que organizan toda la información del sistema.

La tabla de usuarios almacena datos fundamentales como el identificador único, nombre de usuario, correo electrónico, contraseña cifrada, nivel MCER actual, puntos de experiencia acumulados y el rol asignado. El campo rol de la tabla de usuarios distingue entre usuarios estándar y administradores, siendo este valor el que determina el acceso al panel de administración y a los endpoints protegidos de la API.

La tabla de cuestionarios recoge cada pregunta junto con sus cuatro opciones de respuesta, el índice de la opción correcta, el nivel de dificultad y la categoría temática.

La tabla de lecciones contiene todos los recursos didácticos disponibles, especificando el título, una descripción, el tipo de recurso, el contenido textual, la URL del recurso externo y el nivel de aprendizaje asociado.

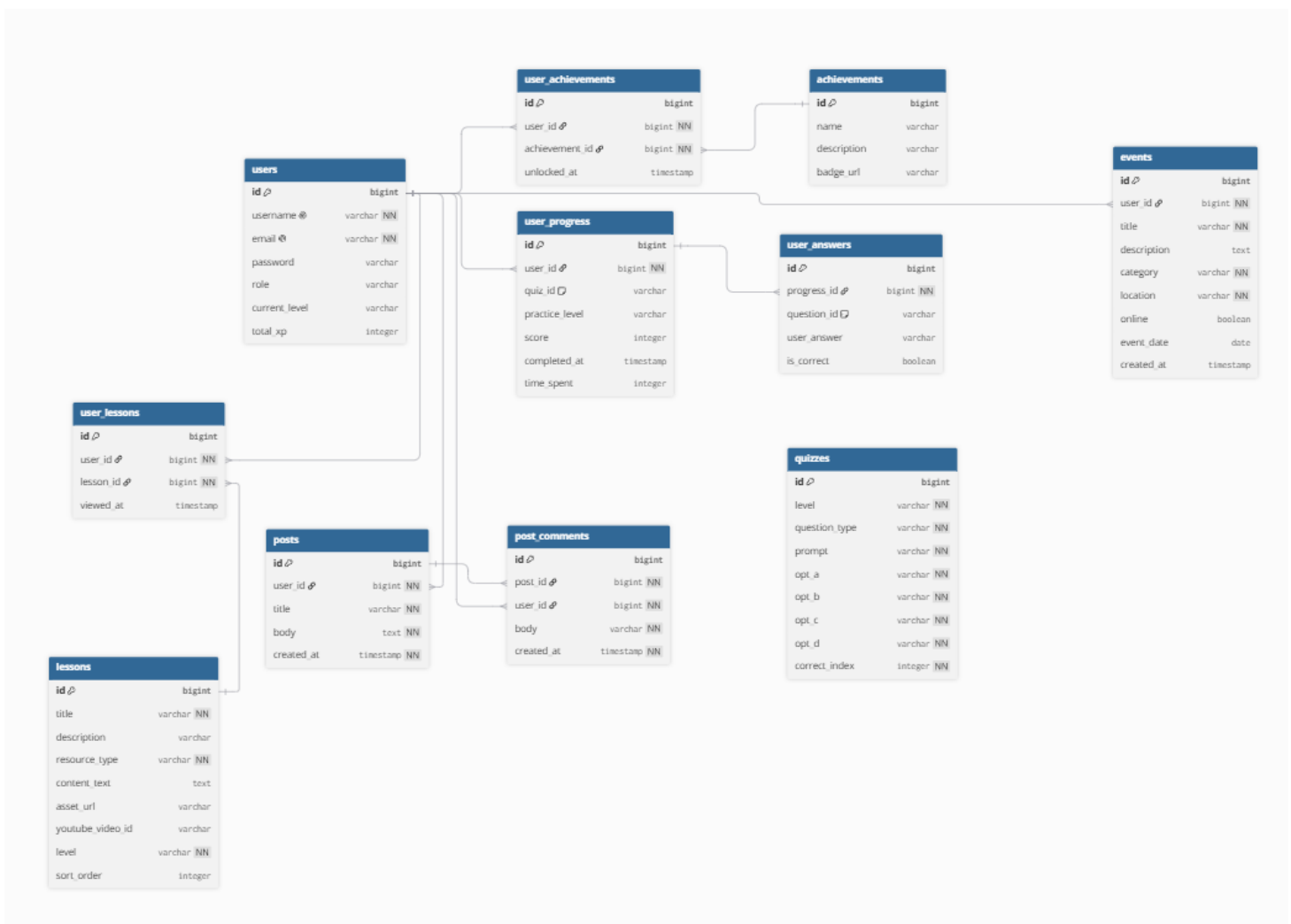
La tabla de lecciones vistas (`user_lessons`) por usuario registra qué recursos ha consultado cada estudiante y en qué momento, vinculando usuarios con lecciones mediante una relación muchos a muchos.

Las tablas de publicaciones y comentarios modelan el blog comunitario con una relación uno a muchos — cada publicación puede tener múltiples comentarios — y ambas están vinculadas a la tabla de usuarios mediante clave foránea.

La tabla de eventos registra los actos de la comunidad con su categoría, fecha, lugar y modalidad de participación, también referenciando al usuario que lo creó.

La tabla user_progress registra cada intento de cuestionario con su puntuación, nivel practicado y tiempo empleado. La tabla user_answers almacena cada respuesta individual, indicando si fue correcta o no. Ambas funcionan como registro histórico de la actividad de cada estudiante.

Finalmente, la tabla de logros almacena todas las recompensas desbloqueables, vinculada a los usuarios a través de una tabla de asociación que registra qué logros ha obtenido cada usuario y cuándo.



5.3 Implementación

La implementación comenzó por el backend. Una vez que se terminaron de desarrollar los endpoints fundamentales de autenticación, cuestionarios y recursos, se desarrollaron en paralelo el frontend web y, más adelante, la aplicación móvil. La integración con los servicios externos de noticias e inteligencia artificial se realizó en una fase posterior, una vez que se consolidó la funcionalidad central de la plataforma.

5.3.1 Backend: API REST con Spring Boot

El backend expone alrededor de veinticinco endpoints REST distribuidos entre ocho controladores.

- El controlador de autenticación gestiona el registro e inicio de sesión generando tokens JWT con duración variable: un día para sesiones estándar y un año con la opción "recordar sesión".
- El controlador de cuestionarios sirve preguntas aleatorias filtradas por nivel y categoría, procesa las respuestas y actualiza la experiencia del usuario otorgando 10 puntos por cada respuesta correcta, según los umbrales definidos por nivel MCER (500 XP para A2, 1500 para B1, 3000 para B2, 5000 para C1 y 8000 para C2). El avance de nivel no es automático: cuando el usuario se aproxima al umbral, la plataforma le indica que está próximo a poder realizar el test oficial de nivel, funcionalidad prevista para una versión futura
- Los controladores de lecciones, publicaciones, eventos y usuarios gestionan el contenido y los perfiles, mientras que el controlador de noticias consulta NewsAPI.org en tiempo real y el de chat delega los mensajes al servicio de IA.

Al arrancar, el sistema ejecuta automáticamente DataLoader y QuizBulkSeeder, que llenan la base de datos con el banco completo de preguntas y lecciones sin intervención manual.

La seguridad se implementa mediante un filtro JWT que intercepta cada petición, valida el token contra la clave secreta del servidor y establece el contexto de autenticación de Spring.

Los endpoints públicos (registro, login, noticias) están explícitamente excluidos de esta validación. La configuración CORS permite el acceso desde el frontend web y la aplicación móvil.

El controlador de administración, accesible únicamente para usuarios con rol `ROLE_ADMIN`, expone endpoints bajo `/api/admin/` para la gestión de usuarios, preguntas, publicaciones y comentarios."

El servicio `GroqService` construye las peticiones al modelo Llama 3.3 con un prompt de sistema diseñado para la enseñanza del inglés, con soporte para respuestas en inglés y español.

5.3.2 Frontend: React con TypeScript

La aplicación web se organiza en nueve páginas gestionadas con `React Router`: registro, login, inicio, cuestionarios, recursos, publicaciones, eventos, perfil y administración, esta última accesible únicamente para usuarios con rol administrador.

Las páginas de registro e inicio de sesión son las encargadas de gestionar el acceso a la plataforma. El login incluye una opción práctica para recordar la sesión, que determina si el token se guarda en `localStorage` (persistiendo entre diferentes visitas del navegador) o en `sessionStorage` (válido únicamente durante esa sesión en particular).

Un módulo cliente centraliza todas las llamadas al backend, inyectando automáticamente el token de autorización en cada petición y ejecutando el cierre de sesión cuando el servidor responde con códigos 401 o 403.

El panel de administración se organiza en cuatro pestañas — Resumen, Usuarios, Comentarios y Preguntas — desde las que el administrador puede gestionar el contenido de la plataforma mediante formularios modales.

El widget de chat con IA se implementa como un componente flotante accesible desde cualquier página de la aplicación, con la excepción de la página de cuestionarios, donde se oculta para no distraer al usuario mientras practica. El widget mantiene un historial completo de la conversación durante toda la sesión y se comunica constantemente con el backend para obtener las respuestas generadas por el modelo de IA.

El panel principal muestra una barra de progreso de XP que refleja el avance del usuario hacia el siguiente nivel MCER. Cuando el usuario alcanza el 80% del umbral, aparece un aviso indicando que está próximo a poder realizar el test de nivel. Al superar el 95%, se muestra el botón *'Take Level Test · Coming soon'* en estado deshabilitado, comunicando que el ascenso de nivel requerirá superar una prueba oficial, funcionalidad prevista para una versión futura.

El diseño visual se construyó utilizando Tailwind CSS, un framework moderno de utilidades CSS que permite estilizar los componentes directamente en el código HTML sin necesidad de escribir hojas de estilo separadas, lo que no solo agiliza significativamente el desarrollo, sino que además garantiza una coherencia visual impecable en toda la aplicación.

5.3.3 Aplicación móvil: React Native con Expo

La aplicación móvil replica de manera inteligente la funcionalidad completa de la versión web, adaptándola específicamente a los patrones naturales de interacción táctil.

La navegación se estructura en dos niveles: un navegador de pila que gestiona el flujo de autenticación, mostrando las pantallas de inicio de sesión y registro a quienes aún no se han autenticado, y un navegador de pestañas inferiores que proporciona acceso a las siete pestañas principales una vez autenticado: Home,

Quiz, Events, Blog, Resources, Profile y Admin, esta última visible únicamente para usuarios con rol administrador.

La sesión se gestiona mediante React Context y el token se almacena de forma cifrada mediante expo-secure-store, restaurándose automáticamente al arrancar la app.

Al igual que en el cliente web, el módulo de llamadas al backend detecta respuestas con códigos 401 o 403 y ejecuta el cierre de sesión de forma automática.

La aplicación incluye además un historial local de intentos de cuestionario implementado mediante expo-file-system, que almacena los resultados en un archivo JSON indexado por el correo del usuario, permitiendo que varios usuarios en el mismo dispositivo mantengan historiales independientes. Se conservan hasta 50 entradas ordenadas de más reciente a más antigua. El archivo persiste entre sesiones, de modo que el historial se recupera automáticamente en el siguiente inicio de sesión del mismo usuario.

A diferencia de la versión web, la aplicación móvil muestra únicamente la barra de XP y el contador de puntos restantes para el siguiente nivel, sin el aviso de proximidad al test de nivel. La aplicación móvil tampoco incluye el asistente conversacional de IA.

5.4 Pruebas

Las pruebas se realizaron de manera continua a lo largo del desarrollo, verificando cada nueva funcionalidad antes de pasar a la siguiente.

Para el backend se utilizó principalmente Postman, creando colecciones de solicitudes que cubrían todos los endpoints disponibles. Se verificaron los flujos de registro e inicio de sesión, la correcta validación de tokens, la lógica de puntuación y ascenso de nivel simulando distintas secuencias de respuestas para asegurar que los umbrales de XP funcionaban correctamente. También se ha validado que las integraciones con NewsAPI.org y Groq funcionaran correctamente.

Las pruebas del frontend se realizaron directamente en el navegador, recorriendo manualmente todos los flujos principales: registro, inicio de sesión con y sin la opción "recordar sesión", cuestionarios completos, acceso a recursos, creación de publicaciones y comentarios, creación de eventos y conversaciones con el asistente. Se enfocó en probar la experiencia cuando algo sale mal, como cuando alguien ingresa credenciales incorrectas o se experimentan problemas de conectividad.

La aplicación móvil se probó en dispositivo físico Android de dos formas: mediante Expo Go, utilizando un túnel ngrok para exponer el servidor de desarrollo y permitir la conexión desde la red móvil, y mediante una APK generada directamente con Expo, instalada en el dispositivo para verificar el comportamiento en un entorno de producción real.

6. Resultados y Producto Final

6.1 Resultados obtenidos

El resultado principal del proyecto es una plataforma de aprendizaje del inglés completamente funcional, accesible tanto desde navegadores web como desde dispositivos móviles. Se alcanzaron los objetivos específicos que se habían planteado al inicio.

El sistema de autenticación basado en JWT demostró funcionar de manera robusta, gestionando sin inconvenientes tanto las sesiones cortas como las persistentes. El módulo de cuestionarios abarcó las cuatro categorías: gramática, vocabulario, modismos (idioms) y comprensión de texto en todos los niveles del MCER, y la lógica de acumulación de XP operó de manera correcta, reflejando el progreso del usuario mediante la barra de XP y el aviso de proximidad al test de nivel.

Los usuarios con rol administrador disponen además de un panel de administración desde el que pueden gestionar usuarios, preguntas, publicaciones y comentarios.

La integración con NewsAPI.org permitió proporcionar contenido auténtico y actualizado en cada visita, mientras que el asistente basado en Llama 3.3 ofreció respuestas coherentes y pedagógicamente útiles.

El entorno completo resultó reproducible localmente en cualquier máquina con Docker instalado, gracias al archivo Docker Compose incluido en el repositorio.

6.2 Descripción del producto final

El producto final es una plataforma web y móvil diseñada para que estudiantes de inglés puedan aprender de manera flexible y personalizada. Cuando se registran indicando su nivel inicial, acceden a un panel intuitivo que muestra su progreso y los invita a practicar de inmediato. Desde la sección de cuestionarios, pueden elegir la categoría que deseen trabajar y responder preguntas de opción múltiple adaptadas a su nivel, obteniendo retroalimentación instantánea y acumulando puntos que les permiten avanzar hacia nuevos niveles.

En la sección de recursos, encontrarán materiales auténticos organizados por nivel: noticias de actualidad para practicar comprensión lectora, letras de canciones, documentos PDF y vídeos seleccionados cuidadosamente. Si surge alguna duda sobre gramática, vocabulario o pronunciación, pueden abrir el widget de chat y consultar con el asistente de IA, que brinda respuestas detalladas y pedagógicas tanto en inglés como en español. La sección comunitaria ofrece la oportunidad de leer y publicar entradas en el blog, además de explorar próximos eventos como charlas, clubes de lectura y sesiones de conversación diseñados específicamente para que los usuarios practiquen inglés en contextos sociales reales. En el perfil personal, se puede consultar en cualquier momento el nivel actual, los puntos acumulados y los logros desbloqueados, lo que mantiene a los estudiantes motivados en su progreso.

La mayor parte de la experiencia está disponible desde la aplicación móvil, a excepción del asistente conversacional de IA, que es exclusivo de la versión web. La navegación mediante pestañas inferiores y el almacenamiento seguro de la sesión

permiten practicar desde cualquier lugar y dispositivo manteniendo la misma funcionalidad que la versión web.

7. Evaluación y Discusión

7.1 Evaluación según objetivos

Los resultados del proyecto respecto a los objetivos iniciales son altamente positivos. Se diseñó y desarrolló una plataforma educativa funcional accesible desde web y móvil. La API REST resultó coherente y bien estructurada, la autenticación JWT funcionó de forma robusta y la configuración CORS permitió el correcto funcionamiento en producción. El sistema de niveles MCER se implementó completamente, incluyendo los seis niveles y la acumulación de puntos de experiencia con indicación visual del progreso mediante barra de XP y aviso de proximidad a la prueba de nivel. Las integraciones con NewsAPI.org y Groq operaron de forma estable. El entorno es reproducible localmente mediante Docker Compose y la plataforma está lista para producción. (si se adquiere dominio).

7.2 Discusión de aciertos y áreas de mejora

La decisión de adoptar una arquitectura API-first fue uno de los aciertos más relevantes, permitiendo que el frontend web y la aplicación móvil se desarrollaran de forma independiente consumiendo los mismos endpoints. Spring Boot fue la elección adecuada por su madurez y su integración nativa con Spring Security y Spring Data JPA. Docker Compose eliminó los problemas de configuración entre máquinas, facilitando la reproducibilidad del entorno.

Pedagógicamente, basar el contenido en recursos auténticos — noticias reales, letras de canciones y vídeos — en lugar de ejercicios prefabricados supuso una diferencia significativa. El asistente de IA basado en Llama 3.3 aportó un valor notable por su capacidad de responder en español y adaptar el nivel de detalle al contexto del usuario.

El área de mejora más evidente es la ausencia de pruebas automatizadas. La incorporación de pruebas unitarias con JUnit en el backend y Testing Library en el frontend habría aumentado la fiabilidad del sistema. También se identificaron oportunidades de mejora en la incorporación de notificaciones push en la aplicación móvil.

7.3 Comparación con expectativas iniciales y estudios previos

Al empezar el proyecto, se estableció una serie de objetivos concretos. En general, los resultados que se obtuvieron se alinearon con lo previsto, aunque con algunos matices. La arquitectura API-first funcionó exactamente como se diseñó al principio: tanto la aplicación web como la móvil consumen los mismos endpoints sin repetir código, lo que hizo el desarrollo de ambas mucho más ágil y eficiente. El sistema de niveles MCER se implementó de forma completa, incluyendo los seis niveles y la progresión automática basada en puntos de experiencia.

Respecto a MongoDB, se mantiene en la arquitectura como capa complementaria preparada para uso futuro, aunque actualmente toda la lógica activa utiliza MySQL.

Por otro lado, se logró implementar el asistente de IA, que al principio no se planteaba en el diseño original.

Las plataformas de aprendizaje de idiomas más consolidadas del mercado - Duolingo y Busuu - comparten algunos objetivos con English Era pero presentan enfoques y limitaciones distintas. English Era se diferencia de ambas plataformas en integrar en un único entorno recursos auténticos de internet - noticias reales de NewsAPI.org, letras de canciones, vídeos y documentos - organizados por nivel MCER, una comunidad activa con publicaciones y eventos presenciales y virtuales, y un asistente conversacional basado en Llama 3.3 que responde en español, aspectos que ninguna de las dos plataformas ofrece de forma conjunta. Como contrapartida, tanto Duolingo como Busuu incluyen práctica oral con reconocimiento de voz y retroalimentación de pronunciación, funcionalidades que English Era no implementa actualmente y que

representan la línea de desarrollo futuro más relevante para completar la experiencia de aprendizaje.

8. Conclusiones

8.1 Principales hallazgos y logros

El desarrollo de English Era ha permitido construir una plataforma educativa completamente funcional y accesible desde navegadores web y dispositivos móviles Android, integrando en un único entorno cuatro componentes que habitualmente se encuentran dispersos: práctica estructurada por niveles MCER, recursos auténticos de internet, una comunidad con publicaciones y eventos, y un asistente conversacional con inteligencia artificial.

Desde el punto de vista técnico, la arquitectura API-first demostró ser la decisión más acertada del proyecto: el frontend web y la aplicación móvil consumen exactamente los mismos endpoints sin duplicar lógica de negocio, lo que permitió desarrollar ambas interfaces de forma independiente y mantener la coherencia de datos en todo momento. La autenticación mediante JWT con duración variable funcionó de manera robusta, y la integración con NewsAPI.org y el modelo Llama 3.3 a través de Groq aportó contenido real y asistencia inteligente sin depender de infraestructura propia costosa.

La aplicación móvil incorpora un sistema de historial local implementado con expo-file-system que almacena hasta 50 resultados de cuestionario por usuario de forma persistente entre sesiones, funcionando incluso sin conexión a internet. Además, se desarrolló un panel de administración completo accesible tanto desde la web como desde la app móvil, que permite gestionar usuarios, preguntas, comentarios y publicaciones en tiempo real. El entorno completo resulta reproducible en cualquier máquina con Docker mediante un único comando, lo que facilita enormemente el despliegue y la colaboración.

8.2 Limitaciones del proyecto

La limitación más significativa es la ausencia de pruebas automatizadas. No se implementaron pruebas unitarias con JUnit en el backend ni pruebas de componentes con Testing Library en el frontend, lo que reduce la fiabilidad del sistema ante cambios futuros y dificulta detectar regresiones de forma sistemática.

El sistema de progresión de nivel presenta una limitación pedagógica importante: aunque el perfil muestra una barra de XP y un aviso de "Level Test — Coming soon" al alcanzar el 95% del umbral, el cambio de nivel no se implementó. La intención original era requerir la superación de una evaluación formal para avanzar de nivel, siguiendo el modelo de certificaciones como Cambridge, pero dicha funcionalidad quedó fuera del alcance del proyecto.

La aplicación móvil presenta además dos limitaciones de paridad respecto a la versión web: no incluye el asistente conversacional de IA, cuyo widget está disponible en la web desde cualquier página, y la pantalla de recursos no consume el endpoint de noticias en tiempo real de NewsAPI.org, mostrando únicamente las lecciones almacenadas en la base de datos y enlaces externos estáticos.

El desarrollo y las pruebas de la aplicación móvil se realizaron exclusivamente sobre Android, mediante Expo Go con túnel ngrok y una APK instalada en dispositivo físico. No se verificó el comportamiento en iOS, por lo que podrían existir diferencias visuales o de comportamiento en esa plataforma. Por último, MongoDB se mantiene en la arquitectura como capa complementaria preparada para uso futuro, pero actualmente no almacena datos activos de la plataforma.

8.3 Recomendaciones

Como línea de trabajo prioritaria se recomienda incorporar una suite de pruebas automatizadas: pruebas unitarias con JUnit 5 y Mockito para los servicios del backend, y

pruebas de integración con MockMvc para los controladores REST. En el frontend, React Testing Library permitiría cubrir los flujos principales de usuario.

En cuanto a la progresión de nivel, se recomienda implementar el test de evaluación formal previsto, idealmente con una batería de preguntas más extensa y equilibrada por categoría, cuyo resultado sea el único criterio para avanzar de nivel, desvinculando así el XP de la certificación de competencia.

Para completar la paridad entre plataformas, se sugiere incorporar el asistente conversacional de IA y las noticias en tiempo real de NewsAPI.org en la aplicación móvil, funcionalidades que ya están implementadas en la versión web y cuya integración en el cliente móvil no requeriría cambios en el backend.

Para la aplicación móvil, se sugiere además verificar la compatibilidad con iOS y añadir notificaciones push mediante Expo Notifications para recordar a los usuarios sus sesiones de práctica diaria. Finalmente, se recomienda definir claramente el papel de MongoDB en la arquitectura: bien adoptarlo para almacenar recursos didácticos con estructura flexible, aprovechando las entidades ya creadas, o bien eliminar la dependencia para simplificar el stack tecnológico.

9. Bibliografía

- Tecnologías y frameworks:
 - Spring Boot. (s.f.). Spring Boot. <https://spring.io/projects/spring-boot>
 - Facebook Inc. (s.f.). React. <https://react.dev>
 - Meta Platforms. (s.f.). React Native. <https://reactnative.dev>
 - Expo. (s.f.). Expo. <https://expo.dev>
- Servicios externos:
 - NewsAPI. (s.f.). NewsAPI.org. <https://newsapi.org>
 - Groq. (s.f.). Groq API. <https://console.groq.com>
 - Meta AI. (s.f.). Llama 3.3. <https://ai.meta.com/blog/llama-3>

- Estándares
 - Consejo de Europa. (s.f.). Marco Común Europeo de Referencia para las Lenguas. <https://www.coe.int/en/web/common-european-framework-reference-languages>
 - OWASP. (s.f.). OWASP Foundation. <https://owasp.org>
- Herramientas:
 - Docker. (s.f.). Docker. <https://www.docker.com>
 - ngrok. (s.f.). ngrok. <https://ngrok.com>
- Otros:
 - La República. (s.f.). El inglés lidera en el mundo: más de 1.500 millones de personas hablan este idioma. <https://www.larepublica.co/ocio/el-ingles-lidera-en-el-mundo-mas-de-1-500-millones-de-personas-hablan-este-idioma-4297797>
 - Ethnologue. (s.f.). Ethnologue: Languages of the world. <https://www.ethnologue.com>
 - Global EdTech. (2024). Busuu launches new AI conversation practice feature. <https://global-edtech.com/busuu-launches-new-ai-conversation-practice-feature/>
 - ToolSmart. (2025). Busuu review. <https://www.toolsmart.ai/blog/busuu-review-plus-the-one-translator-that-complements-it-perfectly/>
 - Duolingo Mania. (2025, febrero 17). How many levels in Duolingo? <https://duolingomania.wordpress.com/2025/02/17/levels-in-duolingo/>
 - Méndez Dunjó, E. (2021). App para aprender inglés de forma interactiva, enfocada al segundo ciclo de primaria [Trabajo de Fin de Grado, Universidad de Las Palmas de Gran Canaria]. Acceda ULPGC. <https://accedacris.ulpgc.es/bitstream/10553/109144/1/TFG%20Eduardo%20Mendez%20Dunjo%202.pdf>

10. Anexos

A. GitHub:

https://github.com/MigleDubauskaite/Migle_TFG_EnglishEra

B. La web

<http://localhost:3000>

C. La enlace de apk:

https://expo.dev/artifacts/eas/PqrAV__GkFe6UWrZyhXBBQLNrB3cNvKiPckthIzjMmY.apk

D. Adminer

<http://localhost:8082/>

Datos para ingresar:

- Sistema: MySQL
- Servidor: mysql-db
- Usuario: root
- Contraseña: root123
- Base de datos: tfg_english_db

MySQL » mysql-db » Base de datos: tfg_english_db

Base de datos: tfg_english_db

[Modificar Base de datos](#) [Esquema de base de datos](#) [Privilegios](#) [Procedimientos](#) [Eventos](#)

Tablas y vistas

Buscar datos en tablas (11)

LIKE %% Condición

☐	Tabla	Motor [?]	Colación [?]	Longitud de datos [?]	Longitud de índice [?]	Espacio libre [?]	Incremento automático [?]	Registros [?]	Comentario [?]
☐	achievements	InnoDB	utf8mb4_0900_ai_ci	16 384	0	0	?	0	
☐	events	InnoDB	utf8mb4_0900_ai_ci	16 384	16 384	0	9	~ 8	
☐	lessons	InnoDB	utf8mb4_0900_ai_ci	16 384	0	0	31	~ 30	
☐	post_comments	InnoDB	utf8mb4_0900_ai_ci	16 384	32 768	0	1	0	
☐	posts	InnoDB	utf8mb4_0900_ai_ci	16 384	16 384	0	3	~ 2	
☐	quizzes	InnoDB	utf8mb4_0900_ai_ci	114 688	0	0	481	~ 480	
☐	user_achievements	InnoDB	utf8mb4_0900_ai_ci	16 384	32 768	0	1	0	
☐	user_answers	InnoDB	utf8mb4_0900_ai_ci	16 384	16 384	0	1	0	
☐	user_lessons	InnoDB	utf8mb4_0900_ai_ci	16 384	32 768	0	1	0	
☐	user_progress	InnoDB	utf8mb4_0900_ai_ci	16 384	16 384	0	1	0	
☐	users	InnoDB	utf8mb4_0900_ai_ci	16 384	32 768	0	3	~ 2	
	11 en total	InnoDB	utf8mb4_0900_ai_ci	278 528	196 608	0			

Selected (0)

[Analizar](#) [Optimizar](#) [Comprobar](#) [Reparar](#) [Vaciar](#) [Eliminar](#)

Mover a otra base de datos (0)

tfg_english_db [Mover](#) [Copiar](#) ☐ overwrite

[Crear tabla](#) [Crear vista](#)

Procedimientos

[Crear procedimiento](#) [Crear función](#)

Eventos

[Crear Evento](#)

E. Swagger:

<http://localhost:8080/swagger-ui/index.html#/>

quiz-controller		^
POST	/api/quizzes/submit	▼
GET	/api/quizzes/stats	▼
GET	/api/quizzes/random	▼
post-controller		^
GET	/api/posts	▼
POST	/api/posts	▼
POST	/api/posts/{id}/comments	▼
GET	/api/posts/{id}	▼
event-controller		^
GET	/api/events	▼
POST	/api/events	▼
chat-controller		^
POST	/api/chat	▼
auth-controller		^
POST	/api/auth/register	▼
POST	/api/auth/login	▼

admin-controller		^
GET	/api/admin/users	▼
POST	/api/admin/users	▼
GET	/api/admin/quizzes	▼
POST	/api/admin/quizzes	▼
DELETE	/api/admin/users/{id}	▼
PATCH	/api/admin/users/{id}	▼
GET	/api/admin/stats	▼
GET	/api/admin/comments	▼
DELETE	/api/admin/quizzes/{id}	▼
DELETE	/api/admin/posts/{id}	▼
DELETE	/api/admin/comments/{id}	▼
user-controller		^
GET	/api/users/me	▼
news-controller		^
GET	/api/news/status	▼
GET	/api/news/live	▼
lesson-controller		^
GET	/api/lessons	▼

F. Base de datos:

